

Experiment No. 6

Experiment Name - Implementation and Comparative Analysis of Image Segmentation Techniques using Python and OpenCV

Steps

NAME : Atul Krishna

COURSE : B.Tech CSE

SECTION - B

SUBJECT : Computer Vision

ROLL NO. : 14

CU ID : CU24250231

1. Import the required Python libraries and load a real-world image such as a medical image, satellite image, or natural scene.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from google.colab import files

uploaded = files.upload()

image_path = list(uploaded.keys())[0]
image = cv2.imread(image_path)

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(4, 3))
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

my photo.jpg
my photo.jpg(image/jpeg) - 12084 bytes, last modified: 8/20/2026 - 100% done
Saving my photo.jpg to my photo (1).jpg

Original Image



2. Convert the image into grayscale and perform basic preprocessing to reduce noise using Gaussian Blur.

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(gray, (5, 5), 0)

plt.figure(figsize=(5, 4))

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(blur, cmap="gray")
plt.title("Gaussian Blurred Image")
plt.axis("off")
plt.show()
```

Grayscale Image

Gaussian Blurred Image



3. Apply Global Thresholding to separate foreground objects from the background and observe the segmentation results.

```
_, global_thresh = cv2.threshold(blur, 127, 255, cv2.THRESH_BINARY)

plt.figure(figsize=(4, 3))
plt.imshow(global_thresh, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")
plt.show()
```

Global Thresholding



4. Implement Otsu's Thresholding to automatically determine the optimal threshold value and compare it with global thresholding.

```
otsu_value, otsu_thresh = cv2.threshold(
    blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

print("Optimal Otsu Threshold Value:", otsu_value)

plt.figure(figsize=(4, 3))
plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu's Thresholding")
plt.axis("off")
plt.show()
```

Optimal Otsu Threshold Value: 155.0

Otsu's Thresholding



5. Apply Adaptive Thresholding to segment images with uneven illumination conditions.

```
adaptive_thresh = cv2.adaptiveThreshold(
    blur,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)
```

```
plt.figure(figsize=(4, 3))
plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")
plt.show()
```

Adaptive Thresholding



6. Implement the Watershed Segmentation algorithm to separate overlapping or touching objects within the image.

```
# Convert threshold image to binary
_, binary = cv2.threshold(
    blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

# Remove noise
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel, iterations=2)

# Sure background
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# Sure foreground
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)

_, sure_fg = cv2.threshold(
    dist_transform,
    0.5 * dist_transform.max(),
    255,
    0
)

sure_fg = np.uint8(sure_fg)

# Unknown region
unknown = cv2.subtract(sure_bg, sure_fg)

# Marker labeling
_, markers = cv2.connectedComponents(sure_fg)

# Add 1 so background becomes 1
markers = markers + 1

# Mark unknown region as 0
markers[unknown == 255] = 0

# Apply Watershed
watershed_image = image.copy()
markers = cv2.watershed(watershed_image, markers)

# Mark boundaries in red
watershed_image[markers == -1] = [0, 0, 255]

watershed_rgb = cv2.cvtColor(watershed_image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(4, 3))
plt.imshow(watershed_rgb)
plt.title("Watershed Segmentation")
plt.axis("off")
plt.show()
```

Watershed Segmentation



7. Apply K-Means Clustering for color-based image segmentation by grouping pixels into distinct clusters.

```
# Convert image from BGR to RGB
rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Reshape pixels
pixels = rgb_image.reshape((-1, 3))
pixels = np.float32(pixels)

# Number of clusters
k = 3

# Apply K-Means
kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

labels = kmeans.fit_predict(pixels)

# Get cluster centers
centers = np.uint8(kmeans.cluster_centers_)

# Replace pixels with cluster centers
segmented_pixels = centers[labels]

# Reshape back to image
segmented_image = segmented_pixels.reshape(rgb_image.shape)

plt.figure(figsize=(4, 3))
plt.imshow(segmented_image)
plt.title("K-Means Segmentation (K = 3)")
plt.axis("off")
plt.show()
```

K-Means Segmentation (K = 3)



8. Compare the segmentation outputs obtained from different techniques based on object boundaries, accuracy, and computational efficiency.

```
plt.figure(figsize=(6, 4))

plt.subplot(2, 3, 1)
plt.imshow(rgb_image)
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(global_thresh, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu Thresholding")
```

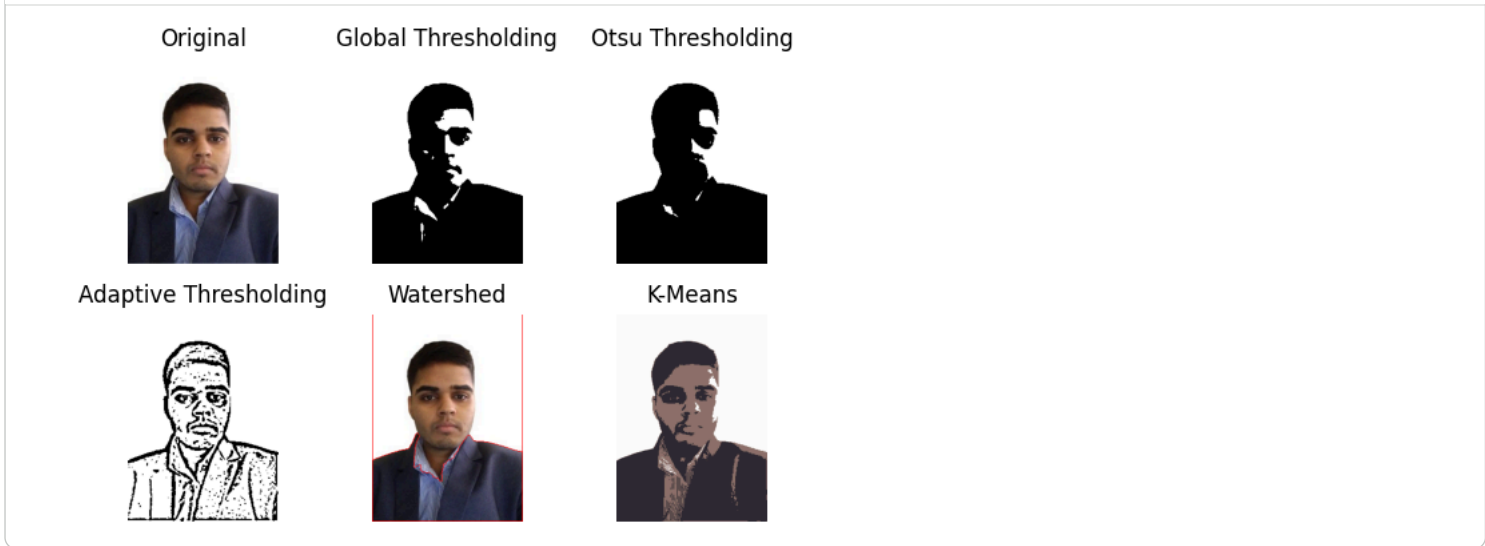
```
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(watershed_rgb)
plt.title("Watershed")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(segmented_image)
plt.title("K-Means")
plt.axis("off")

plt.tight_layout()
plt.show()
```



9. Visualize and analyze the segmented images to identify the most suitable technique for the selected dataset.

Analysis of Segmentation Results

The segmentation results were analyzed based on object boundaries, accuracy, and computational efficiency.

- **Global Thresholding:** It is simple and computationally fast, but its performance depends on a fixed threshold value and may not work well with uneven illumination.
- **Otsu's Thresholding:** It automatically selects an optimal threshold value and generally provides better segmentation when the foreground and background have different intensity distributions.
- **Adaptive Thresholding:** It performs better when the image has uneven lighting because the threshold is calculated for different local regions of the image.
- **Watershed Segmentation:** It is useful for detecting boundaries and separating touching or overlapping objects. However, it may require proper preprocessing and marker selection.
- **K-Means Clustering:** It is effective for color-based segmentation by grouping pixels with similar color characteristics. The result depends on the selected number of clusters (K).

Conclusion of Analysis

Among the implemented techniques, no single method is best for every image. The most suitable segmentation technique depends on the image characteristics such as intensity, color, illumination, and object boundaries.

10. Summarize the observations and discuss the role of image segmentation in advanced computer vision applications.

Summary and Conclusion

Summary

Image segmentation is an important technique in computer vision that divides an image into meaningful regions or objects. In this experiment, different segmentation techniques such as Global Thresholding, Otsu's Thresholding, Adaptive Thresholding, Watershed Segmentation, and K-Means Clustering were implemented using Python and OpenCV.

Observations

- **Global Thresholding** is simple and fast but works best when the foreground and background have clear intensity differences.
- **Otsu's Thresholding** automatically selects a suitable threshold and reduces the need for manual threshold selection.

- **Adaptive Thresholding** is useful for images with uneven illumination because it calculates the threshold locally.
- **Watershed Segmentation** is useful for detecting boundaries and separating touching or overlapping objects.
- **K-Means Clustering** performs color-based segmentation by grouping pixels with similar color characteristics.

Conclusion

The experiment shows that different image segmentation techniques are suitable for different types of images and applications. Thresholding methods are useful for intensity-based segmentation, Watershed is effective for separating connected objects, and K-Means is useful for color-based segmentation.

Image segmentation plays an important role in advanced computer vision applications such as object detection, medical image analysis, autonomous navigation, satellite image analysis, and industrial inspection. Therefore, selecting an appropriate segmentation technique according to the image characteristics is essential for obtaining accurate and useful results.

Questions and Answers

1. What is image segmentation? Why is it considered a fundamental step in computer vision?

Image segmentation is the process of dividing an image into meaningful regions or objects based on properties such as pixel intensity, color, texture, or boundaries.

It is considered a fundamental step in computer vision because it separates the **Region of Interest (ROI)** from the background and makes further tasks such as object detection, recognition, tracking, and image analysis easier and more accurate.

Example: Separating a person's face from the background in an image.

2. Differentiate between Image Segmentation and Image Classification.

Image Segmentation	Image Classification
Divides an image into meaningful regions or objects.	Assigns a class or label to an entire image.
Works at the pixel level.	Usually works at the image level.
Identifies the location and boundaries of objects.	Identifies what the image represents.
Output is a segmented image or mask.	Output is a class label.
Example: Separating a person from the background.	Example: Classifying an image as "Cat" or "Dog".

In simple words: Classification tells **what is present**, while segmentation tells **what is present and where it is located**.

3. Explain the working principle of Global Thresholding, Otsu's Thresholding, and Adaptive Thresholding.

(a) Global Thresholding

Global Thresholding uses a single fixed threshold value to divide an image into foreground and background.

If the pixel intensity is greater than the threshold, it is assigned to one class; otherwise, it is assigned to the other class.

Example: Pixels > 127 → White, Pixels ≤ 127 → Black.

(b) Otsu's Thresholding

Otsu's Thresholding automatically selects the optimal threshold value from the image histogram.

It tries to find a threshold that gives the best separation between foreground and background.

Advantage: No need to manually select the threshold value.

(c) Adaptive Thresholding

Adaptive Thresholding calculates a different threshold for different local regions of an image.

It is useful when the image has **uneven illumination or varying brightness**.

In simple words:

- Global → One fixed threshold for the whole image.
- Otsu → Automatically finds one optimal threshold.
- Adaptive → Uses different thresholds for different regions.

4. What is the Watershed Algorithm? Why is it useful for separating overlapping objects?

The Watershed Algorithm is an image segmentation technique that treats the grayscale image like a **topographic surface** and identifies boundaries between different regions.

It is especially useful for separating **touching or overlapping objects** because it detects boundaries between connected regions.

Working:

1. Convert the image into a suitable binary form.
2. Identify the sure background and sure foreground.

3. Create markers for different regions.
4. Apply the Watershed algorithm.
5. The boundaries between different regions are identified.

Example: Separating two touching coins or cells in a medical image.

5. How is K-Means Clustering applied to image segmentation?

Answer:

K-Means Clustering is an unsupervised machine learning technique that groups pixels into different clusters based on their similarity.

For image segmentation, the pixel values such as **RGB color values** are used as input to K-Means.

Working:

1. Select the number of clusters (**K**).
2. Assign initial cluster centers.
3. Assign each pixel to the nearest cluster center.
4. Recalculate the cluster centers.
5. Repeat the process until the clusters become stable.
6. Replace each pixel with the value of its cluster center.

Example: If (**K** = 3), the image can be divided into three color-based regions.

6. Compare threshold-based segmentation and clustering-based segmentation techniques.

Threshold-Based Segmentation	Clustering-Based Segmentation
Divides pixels using threshold values.	Divides pixels into groups or clusters.
Mainly depends on pixel intensity.	Can use color, intensity, or other features.
Simple and computationally fast.	Usually more computationally expensive.
Suitable for images with clear foreground-background differences.	Suitable for images containing multiple regions or colors.
Examples: Global, Otsu, Adaptive Thresholding.	Example: K-Means Clustering.

In simple words: Thresholding separates pixels using **threshold values**, while clustering groups similar pixels into **clusters**.

7. What challenges are encountered while segmenting images with complex backgrounds or varying illumination?

Some major challenges are:

- **Uneven illumination:** Different parts of the image may have different brightness levels.
- **Complex backgrounds:** The background may contain colors or patterns similar to the object.
- **Noise:** Random variations in pixels can produce incorrect segmentation.
- **Low contrast:** Foreground and background may have very similar intensities.
- **Object overlap:** Multiple objects may touch or overlap with each other.
- **Shadows and reflections:** They can make object boundaries difficult to identify.

These challenges can reduce the accuracy of segmentation and may require preprocessing or advanced segmentation methods.

8. Mention five real-world applications where image segmentation plays a critical role.

Five important applications of image segmentation are:

1. **Medical Imaging** — Detecting tumors, organs, and abnormal regions.
2. **Autonomous Vehicles** — Separating roads, vehicles, pedestrians, and obstacles.
3. **Satellite Image Analysis** — Identifying buildings, roads, forests, and water bodies.
4. **Object Detection and Recognition** — Separating objects from their background.
5. **Industrial Inspection** — Detecting defects and damaged regions in manufactured products.

9. How does image segmentation improve the performance of object detection and image recognition systems?

Image segmentation separates the important objects or **Region of Interest (ROI)** from the background.

This reduces unnecessary background information and allows the computer vision system to focus on the relevant objects.

As a result, segmentation can:

- Improve object localization.
- Provide accurate object boundaries.
- Reduce background noise.
- Improve feature extraction.
- Increase recognition and detection accuracy.
- Make further image analysis easier.

In simple words: Image segmentation helps the system focus on the **important parts of an image**, which can improve object detection and recognition performance.

10. Compare traditional image segmentation techniques with deep learning-based segmentation methods such as U-Net and Mask R-CNN.

Traditional Segmentation	Deep Learning-Based Segmentation
Uses manually designed rules or algorithms.	Learns segmentation features automatically from data.
Examples: Thresholding, Watershed, K-Means.	Examples: U-Net and Mask R-CNN.
Usually requires less training data.	Usually requires a large labeled dataset for training.
Computationally simpler in many cases.	Requires more computational resources and training.
Performance can decrease with complex images.	Generally performs better on complex images when properly trained.
Does not require model training in many cases.	Requires training or a pretrained model.
Suitable for simple and controlled environments.	Suitable for complex real-world segmentation tasks.

U-Net

U-Net is a deep learning architecture mainly used for **semantic segmentation**. It is widely used in medical image segmentation because it can accurately identify regions or structures in an image.

Mask R-CNN

Mask R-CNN is a deep learning-based method that performs **instance segmentation**. It detects individual objects and generates a separate segmentation mask for each object.

Conclusion

Traditional methods are simple, fast, and useful for basic segmentation tasks, while deep learning methods such as **U-Net and Mask R-CNN** are more powerful for complex images and real-world applications but require more data and computational resources.